

SP4 - Mission 1 : Mise en place d'un environnement de test conteneurisé



Table des matières

Docker :.....	3
Installation de Docker et Docker compose :.....	3
Vérifier le fonctionnement de Docker :.....	3
Mettre une interface web pour Docker :.....	4
Créer des conteneurs pour différents services :.....	5
Service web :.....	5
Base de données :.....	6
Mettre en place un système de versionning avec github :.....	7

Docker :

Installation de Docker et Docker compose :

Sur votre Linux **Debian** ou **Ubuntu**, commencer par mettre à jour votre machine.

```
etudiant@debian:~$ sudo apt update && sudo apt upgrade -y
```

Vous pouvez ensuite installer **Docker** et **Docker compose**.

```
etudiant@debian:~$ sudo apt-get install docker
etudiant@debian:~$ sudo apt-get install docker-compose
```

Vérifier le fonctionnement de Docker :

Commencer par vérifier la **version de Docker** et **Docker-compose**.

```
etudiant@debian:~$ docker-compose --version
docker-compose version 1.29.2, build unknown
etudiant@debian:~$ docker --version
Docker version 20.10.24+dfsg1, build 297e128
```

Pour être sûr de son fonctionnement, créer un conteneur simple.

```
etudiant@debian:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:452a468a4bf985040037cb6d5392410206e47db9bf5b7278d281f94d1c2d0931
Status: Downloaded newer image for hello-world:latest
```

Mettre une interface web pour Docker :

Créer le dossier qui contiendra vos conteneurs.

```
etudiant@debian:~$ sudo mkdir -p /root/docker/portainer
```

/root → Répertoire du superutilisateur

/docker → Contient les conteneurs

/portainer → Conteneur portainer

Portainer est l'interface web que nous utiliserons pour Docker.

Dans docker créer un fichier docker-compose.yml, celui-ci va contenir les configurations de vos conteneurs, modifier le avec les informations suivante.

version: "3.9"

services:

portainer:

image: portainer/portainer-ce:latest

container_name: portainer

restart: always

ports:

- "9000:9000"

volumes:

- /var/run/docker.sock:/var/run/docker.sock

- portainer_data:/data

volumes:

portainer_data:

Vous pouvez ensuite utiliser la commande suivante pour faire fonctionner le **fichier .yml**.

```
etudiant@debian:~/docker$ docker-compose up -d
```

Sur la page d'un navigateur de recherche mettre :

<http://ip-du-serveur-docker:9000>

9000 → Port du conteneur.

Créer des conteneurs pour différents services :

Service web :

Dans le dossier Docker créer le dossier **html** qui est donc le conteneur pour le Service web.

```
etudiant@debian:~$ sudo mkdir /home/etudiant/docker/html
```

Dans le fichier **docker-compose.yml** du dossier **Docker** ajouter les lignes suivantes pour le Service web.

services:

web:

image: php:8.2-apache

ports:

- "8080:80"

volumes:

- ./site:/var/www/html

depends_on:

- db

Dans le dossier **html** créer un fichier **html.index** avec un petit message dedans pour vérifier le fonctionnement du conteneur.

Vous pouvez ensuite utiliser la commande suivante pour faire fonctionner le **fichier .yml** avec l'ajout du Service web.

```
etudiant@debian:~/docker$ docker-compose up -d
```

Sur la page d'un navigateur de recherche mettre :

http://ip-du-serveur-docker:8080

8080 → Port du conteneur.



Base de données :

Dans le dossier Docker créer le dossier **mysql** qui est donc le conteneur pour la base de données.

```
etudiant@debian:~$ sudo mkdir /home/etudiant/docker/mysql
```

Dans le fichier **docker-compose.yml** du dossier **Docker** ajouter les lignes suivantes pour la base de données.

db:

image: mysql:8

restart: always

environment:

MYSQL_ROOT_PASSWORD: root

MYSQL_DATABASE: monsite

MYSQL_USER: user

MYSQL_PASSWORD: password

ports:

- "3306:3306"

volumes:

- ./database:/docker-entrypoint-initdb.d

Vous pouvez ensuite utiliser la commande suivante pour faire fonctionner le **fichier .yml** avec l'ajout du Service web.

```
etudiant@debian:~/docker$ docker-compose up -d
```

Sur le dossier du conteneur **mysql** directement, connecter vous à la bd pour vérifier le bon fonctionnement.

```
etudiant@debian:~/docker/mysql$ sudo docker exec -it mysql_db mysql -uAdmMysql -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> █
```

Mettre en place un système de versionning avec github :

Installer git sur votre docker.

```
etudiant@debian:~$ sudo apt update && sudo apt install git && sudo apt upgrade
```

Configurer son identité Git.

```
etudiant@debian:~$ git config --global user.name "Boisseau"  
etudiant@debian:~$ git config --global user.email "crys.boisseau@gmail.com"
```

Diriger vous vers le dossier contenant vos containers, dans notre cas docker et initialiser Git.

```
etudiant@debian:~$ cd docker/  
etudiant@debian:~/docker$ git init  
astuce: Utilisation de 'master' comme nom de la branche initiale. Le nom de la branche  
astuce: par défaut peut changer. Pour configurer le nom de la branche initiale  
astuce: pour tous les nouveaux dépôts, et supprimer cet avertissement, lancez :  
astuce:  
astuce:         git config --global init.defaultBranch <nom>  
astuce:  
astuce: Les noms les plus utilisés à la place de 'master' sont 'main', 'trunk' et  
astuce: 'development'. La branche nouvellement créée peut être renommée avec :  
astuce:  
astuce:         git branch -m <nom>  
Dépôt Git vide initialisé dans /home/etudiant/docker/.git/  
etudiant@debian:~/docker$
```

Créer un fichier .gitignore en y ajoutant le texte ci-dessous afin d'éviter d'envoyer des informations sensible.

```
etudiant@debian:~/docker$ touch .gitignore  
etudiant@debian:~/docker$ sudo nano .gitignore
```

Docker

docker-data/

***.log**

PHP

vendor/

Env

.env

IDE

.vscode/

.idea/

OS

.DS_Store

Sur github, créer un répertoire pour votre docker et exécuter les commandes suivantes sur votre docker ensuite.

```
etudiant@debian:~/docker$ git add .
etudiant@debian:~/docker$ git commit -m "Initialisation du projet"
etudiant@debian:~/docker$ git remote add origin https://github.com/crysboisseau/docker_versionning.git
etudiant@debian:~/docker$ git branch -M main
etudiant@debian:~/docker$ git push -u origin main
```

- **git add .** : Ajouter tous les fichiers et dossiers.
- **git commit** : Commentaires.
- **git remote add origin** : Choisi votre répertoire sur github.
- **git branch -M main** : Sélectionne la branche main.
- **git push -u origin main** : Envoie les dossiers et fichiers dans votre répertoire github

Le nom d'utilisateur de votre compte github va être demandé, et un password, vous pouvez générer un token sur github qui vous servira de mdp.